

COMPTE RENDU

Mini Projet — Java Avancée / Architectures Logicielles

Établissement	ISG Bizerte
Module	Java Avancée & Architectures Logicielles
Enseignantes	Wafa Neji Stancati & Hela Jabbar
Membres du groupe	Mariam Khcharem, Lemis Zameli, Dhouha Hajri
Classe	2LIG2
Année universitaire	2025 – 2026

1. Introduction

Ce compte rendu présente le travail réalisé dans le cadre du mini-projet des modules Java Avancée et Architectures Logicielles. L'objectif était de concevoir et développer un microservice de gestion scolaire en utilisant Spring Boot et Spring Data JPA, en respectant une architecture en couches.

Le projet modélise six entités JPA — School, Departement, Student, Adress, Instructor et Course — reliées par des associations de différentes cardinalités et navigabilités. La base de données utilisée est PostgreSQL, et les endpoints REST ont été testés via Postman.

2. Structure du projet

Le projet suit la structure standard Maven d'une application Spring Boot, organisée en cinq packages principaux selon le principe de séparation des responsabilités :

- — **classe principale SchoolApplication, point d'entrée de l'application.**edu.isgb.school
- — **les six entités JPA représentant les tables de la base de données.**edu.isgb.school.entities
- — **les interfaces Spring Data JPA pour l'accès aux données.**edu.isgb.school.repository
- — **la classe SchoolService contenant toute la logique métier.**edu.isgb.school.service
- — **le contrôleur REST TestSchoolController exposant les endpoints.**edu.isgb.school.test

Le fichier application.properties configure la connexion à PostgreSQL ainsi que le comportement de Hibernate :

```
spring.datasource.url=jdbc:postgresql://localhost:5432/schooldb

spring.datasource.username=postgres

spring.datasource.password=[ mot de passe ]


spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.format_sql=true
```

La propriété ddl-auto=update permet à Hibernate de créer et mettre à jour automatiquement les tables sans supprimer les données existantes. show-sql=true affiche les requêtes SQL générées dans la console IntelliJ, et format_sql=true les formate de façon lisible.

3. Architecture en couches

L'application respecte le modèle d'architecture 3-tiers enseigné en cours, garantissant une séparation claire des responsabilités entre les différentes couches :

Couche	Classe / Interface	Rôle
Présentation	TestSchoolController	Reçoit les requêtes HTTP, extrait les données (JSON, paramètres URL), délègue au service et retourne la réponse JSON.
Métier	SchoolService	Contient toute la logique applicative. Orchestre les opérations entre entités, gère les transactions avec <code>@Transactional</code> .
Accès aux données	Repositories JPA	Interfaces étendant <code>JpaRepository</code> . Génèrent automatiquement les opérations CRUD et les requêtes SQL sans code SQL manuel.
Base de données	PostgreSQL (schooldb)	Stocke les données de façon persistante. Les tables sont générées et maintenues automatiquement par Hibernate.

4. Entités JPA et mapping Objet-Relationnel

Conformément au cours sur JPA, chaque entité est une classe Java annotée `@Entity`, correspondant à une table de la base de données. L'ORM (Object Relational Mapping) établit la correspondance entre les objets Java et les enregistrements de la base de données, éliminant la nécessité d'écrire du SQL manuellement.

Les annotations Lombok (`@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`) réduisent le code répétitif en générant automatiquement les getters, setters, et constructeurs à la compilation.

4.1 Entité School

La classe `School` est l'entité centrale du projet. Elle est mappée avec la table `t_school` et entretient des relations bidirectionnelles avec `Departement`, `Instructor` et `Student`.

```

@Entity

@Table(name = "t_school")

@Data @NoArgsConstructor @AllArgsConstructor

public class School implements Serializable {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    @Column(name = "PK_school")

    private Long id;

    @Column(name = "cl_name_school", nullable = false)

    private String name;

    private String phone;

    @OneToMany(mappedBy = "school", cascade = CascadeType.ALL, fetch = FetchType.LAZY)

    private List<Departement> departments = new ArrayList<>();

    // Idem pour instructors et students

}

```

L'annotation `@Column(name = "PK_school")` permet de spécifier un nom de colonne différent du nom du champ Java. L'annotation `mappedBy = "school"` indique que c'est `Departement` qui possède la clé étrangère dans la base de données — `School` est le côté passif de la relation bidirectionnelle.

4.2 Entité `Departement`

```

@Entity

@Table(name = "t_departement")

public class Departement implements Serializable {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    @Column(name = "pk_department")

    private Long id;

    @Column(name = "cl_name", nullable = false)

    private String name;

    @ManyToOne

    @JoinColumn(name = "school_PK_school")

    @JsonIgnore

    private School school;

}

```

L'annotation `@JoinColumn(name = "school_PK_school")` crée la colonne de clé étrangère dans la table `t_departement`. C'est le côté "owning" (propriétaire) de la relation : c'est lui qui génère physiquement la FK en base de données. L'annotation `@JsonIgnore` évite la boucle infinie lors de la sérialisation JSON (School → ses Departements → leur School → ...).

4.3 Entité Student

```

@Entity

@Table(name = "t_student")

public class Student implements Serializable {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;


    private String firstName;

    private String lastName;


    @Column(name = "cl_birthdate")

    private Date birthDate;


    @ManyToOne

    @JoinColumn(name = "school_id")

    @JsonIgnore

    private School school;


    @OneToOne(cascade = CascadeType.ALL)

    @JoinColumn(name = "address_id")

    private Adress address;

}

```

Student entretient deux relations distinctes : une relation ManyToOne bidirectionnelle avec School (plusieurs étudiants appartiennent à une école), et une relation OneToOne unidirectionnelle avec Adress (un étudiant possède une adresse, mais l'adresse ne référence pas l'étudiant). Le CascadeType.ALL sur la relation avec Adress lie le cycle de vie des deux entités : sauvegarder un étudiant sauvegarde automatiquement son adresse.

4.4 Entité Adress

```
@Entity

@Table(name = "t_adress")

public class Adress implements Serializable {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String street;

    private String city;

    private String zipCode;

    // Pas de champ Student : mapping UNIDIRECTIONNEL

}
```

L'absence de champ référençant Student confirme le mapping unidirectionnel imposé par le sujet. Adress est un objet de valeur dont le cycle de vie est entièrement géré par Student via le cascade.

4.5 Entité Instructor

```

@Entity

@Table(name = "t_instructor")

public class Instructor implements Serializable {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String name;

    @ManyToOne

    @JoinColumn(name = "school_id")

    @JsonIgnore

    private School school;

    @OneToMany(mappedBy = "instructor", cascade = CascadeType.ALL, fetch = FetchType.LAZY)

    private List<Course> courses = new ArrayList<>();

    public void addCourse(Course c) {

        courses.add(c);

        c.setInstructor(this);

    }

}

```

La méthode helper `addCourse()` est indispensable pour les relations bidirectionnelles : elle synchronise les deux côtés de la relation avant la sauvegarde. Sans cette synchronisation, la colonne `instructor_id` resterait NULL en base de données même si la liste en mémoire est remplie, car JPA ne gère la FK que du côté owning (Course).

4.6 Entité Course

```
@Entity

@Table(name = "t_course")

public class Course implements Serializable {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String name;

    @ManyToOne

    @JoinColumn(name = "instructor_id")

    @JsonIgnore

    private Instructor instructor;

}
```

5. Analyse des cas de mapping et justification des choix

5.1 Question 2 — Relation School ↔ Departement (OneToMany)

Le sujet demande de tester trois cas de mapping pour la relation entre School et Departement, et de justifier le choix le plus adéquat.

Cas	Description technique	Schéma DB généré	Verdict
Cas 1	Bidirectionnel sans @JoinColumn côté Departement	Trois tables : t_school, t_departement, et une table de jointure t_school_t_departement avec deux colonnes FK.	Non retenu
Cas 2	Bidirectionnel avec @JoinColumn(name="school_PK_school") dans Departement et mappedBy="school" dans School	Deux tables uniquement : la FK school_PK_school est stockée directement dans t_departement.	Retenu ✓
Cas 3	Unidirectionnel School → Departement uniquement	Deux tables avec FK dans t_departement (schéma identique au Cas 2), mais navigation inverse impossible.	Non retenu

Justification du choix : Cas 2

Le Cas 2 est le schéma le plus adéquat pour trois raisons principales.

Premièrement, il évite la table de jointure superflue générée par le Cas 1. Dans le modèle relationnel standard, une relation OneToMany est représentée par une clé étrangère dans la table du côté "many" (Departement), et non par une table intermédiaire. Une table de jointure est utile uniquement pour les relations ManyToMany.

Deuxièmement, le mapping bidirectionnel permet une navigation dans les deux sens : on peut obtenir tous les départements d'une école via `school.getDepartments()`, et retrouver l'école d'un département via `dept.getSchool()`. Le Cas 3 (unidirectionnel) ne permet que la première navigation, ce qui est insuffisant pour les besoins de l'application.

Troisièmement, le schéma généré correspond exactement au schéma imposé par la Figure 5 du sujet, qui montre explicitement la colonne `school_PK_school` dans `t_departement`.

5.2 Question 4 — Relation Student → Adress (OneToOne)

Le sujet impose un mapping unidirectionnel Student → Adresse. Trois cas ont été étudiés.

Cas	Description technique	Schéma DB généré	Verdict
Cas 1	Unidirectionnel @OneToOne sans @JoinColumn	Table de jointure inutile générée automatiquement par JPA.	Non retenu
Cas 2	Bidirectionnel avec champ Student dans Adress	FK address_id dans t_student + champ student dans t_adress. Couplage inutile.	Non retenu
Cas 3	Unidirectionnel @OneToOne avec @JoinColumn(name="address_id") dans Student uniquement	FK address_id dans t_student. Adress ne connaît pas Student.	Retenu ✓

Justification du choix : Cas 3

Le Cas 3 est retenu pour deux raisons. D'abord, le sujet impose explicitement un mapping unidirectionnel Student → Adresse dans les règles de navigabilité, ce qui exclut le Cas 2. Ensuite, d'un point de vue conception, une adresse est un objet de valeur appartenant à l'étudiant : elle n'a aucune raison métier de référencer l'étudiant qui la possède. Le couplage bidirectionnel du Cas 2 créerait une dépendance inutile.

Le CascadeType.ALL sur la relation @OneToOne assure que le cycle de vie des deux entités est lié : la sauvegarde d'un étudiant entraîne automatiquement la sauvegarde de son adresse, sans gestion manuelle.

5.3 Question 6 — Relation Instructor ↔ Course (OneToMany)

Le sujet impose un mapping bidirectionnel entre Instructor et Course. Deux cas ont été étudiés.

Cas	Description technique	Schéma DB généré	Verdict
Cas 1	Bidirectionnel sans @JoinColumn dans Course	Table de jointure t_instructor_t_course en plus des deux tables principales.	Non retenu
Cas 2	Bidirectionnel avec @JoinColumn(name="instructor_id") dans Course et mappedBy="instructor" dans Instructor	FK instructor_id dans t_course uniquement. Schéma propre sans table intermédiaire.	Retenu ✓

Justification du choix : Cas 2

Le raisonnement est identique à la relation School ↔ Département : le Cas 2 génère un schéma propre sans table de jointure superflue, et permet une navigation bidirectionnelle (instructor.getCourses() et course.getInstructor()). La FK instructor_id dans t_course est conforme au schéma de la Figure 5 du sujet.

L'annotation FetchType.LAZY optimise les performances : les cours d'un instructeur ne sont chargés depuis la base de données que lorsqu'ils sont explicitement demandés, évitant des requêtes SQL inutiles à chaque chargement d'un instructeur.

5.4 Question 7 — Récapitulatif du mapping complet

Le tableau suivant récapitule l'ensemble des règles de navigabilité respectées dans le projet, conformément aux exigences de la Figure 5 du sujet :

Relation	Type	Navigabilité	FK dans	Annotations clés
School ↔ Departement	OneToMany	Bidirectionnelle	t_departement	@ManyToOne @JoinColumn côté Dept / @OneToMany(mappedBy) côté School
School ↔ Instructor	OneToMany	Bidirectionnelle	t_instructor	@ManyToOne @JoinColumn côté Instructor / @OneToMany(mappedBy) côté School
School ↔ Student	OneToMany	Bidirectionnelle	t_student	@ManyToOne @JoinColumn côté Student / @OneToMany(mappedBy) côté School
Student → Adress	OneToOne	Unidirectionnelle	t_student	@OneToOne(cascade=ALL) @JoinColumn côté Student uniquement
Instructor ↔ Course	OneToMany	Bidirectionnelle	t_course	@ManyToOne @JoinColumn côté Course / @OneToMany(mappedBy) côté Instructor

6. Repositories Spring Data JPA

Conformément à l'architecture en couches, chaque entité dispose d'une interface repository dans le package `edu.isgb.school.repository`. Ces interfaces étendent `JpaRepository<T, ID>`, ce qui leur fournit automatiquement toutes les opérations CRUD standard sans écrire une seule ligne de SQL.

Interface	Entité gérée	Méthodes supplémentaires
SchoolRepository	School	Aucune (CRUD de base suffit)
DepartementRepository	Departement	Aucune
StudentRepository	Student	Aucune
AdressRepository	Adress	Aucune
InstructorRepository	Instructor	<code>findByNameContaining(String name)</code>
CourseRepository	Course	Aucune

La méthode `findByNameContaining(String name)` dans `InstructorRepository` est une Query Method dérivée : Spring Data JPA analyse automatiquement le nom de la méthode et génère la requête SQL correspondante (`SELECT * FROM t_instructor WHERE name LIKE '%name%'`) sans aucune annotation ni code SQL.

7. Service — SchoolService

La classe `SchoolService`, annotée `@Service`, contient l'ensemble de la logique métier du projet. Elle s'appuie sur les repositories pour l'accès aux données et gère les transactions avec l'annotation `@Transactional`.

7.1 Méthode `createSchoolWithDetails` (8a)

```

@Transactional

public School createSchoolWithDetails(School school) {

    for (Student s : school.getStudents()) s.setSchool(school);

    for (Instructor i : school.getInstructors()) i.setSchool(school);

    for (Departement d : school.getDepartments()) d.setSchool(school);

    return schoolRepo.save(school);

}

```

Les boucles for synchronisent manuellement les clés étrangères bidirectionnelles avant la sauvegarde. Sans cette étape, les colonnes `school_id` et `school_PK_school` resteraient NULL en base de données. Le `CascadeType.ALL` sur `School` propage automatiquement le `save()` à tous les enfants (départements, instructeurs, étudiants).

7.2 Méthode `createStudentWithAddressAndSchool` (8c)

```

@Transactional

public Student createStudentWithAddressAndSchool(Student student, Long schoolId) {

    School school = getSchoolById(schoolId);

    student.setSchool(school);

    return studentRepo.save(student);

}

```

L'adresse est sauvegardée automatiquement avec l'étudiant grâce au `CascadeType.ALL` sur la relation `@OneToOne`. Il n'est pas nécessaire de sauvegarder l'adresse manuellement via `addressRepo`.

7.3 Méthode `getCoursesByInstructorId` (8i)

```
@Transactional

public List<Course> getCoursesByInstructorId(Long instructorId) {

    Instructor instructor = getInstructorById(instructorId);

    return instructor.getCourses();

}
```

L'annotation `@Transactional` est indispensable sur cette méthode. Les cours sont configurés en `FetchType.LAZY`, ce qui signifie qu'ils ne sont pas chargés automatiquement avec l'instructeur. Sans `@Transactional`, la session JPA serait fermée avant que la liste ne soit accédée, provoquant une `LazyInitializationException`. `@Transactional` maintient la session ouverte pendant toute la durée de la méthode.

7.4 Récapitulatif de toutes les méthodes

Méthode	Question	Description
<code>createSchoolWithDetails(school)</code>	8a	Crée une School avec ses Students, Instructors et Departments en une seule transaction.
<code>getSchoolById(id)</code>	8b	Retourne une School par son id. Lève une <code>RuntimeException</code> si non trouvée.
<code>createStudentWithAddressAndSchool(student, schoolId)</code>	8c	Crée un Student avec son Address et l'associe à une School existante.
<code>listAllStudents()</code>	8d	Retourne la liste complète de tous les Students via <code>findAll()</code> .
<code>createInstructorWithCourses(instructor)</code>	8e	Crée un Instructor avec sa liste de Courses en synchronisant les FK.
<code>listInstructorsByName(name)</code>	8f	Recherche les Instructors dont le nom contient la chaîne donnée.
<code>getInstructorById(id)</code>	8g	Retourne un Instructor par son id.
<code>getCourseById(id)</code>	8h	Retourne un Course par son id.
<code>getCoursesByInstructorId(instructorId)</code>	8i	Liste les Courses d'un Instructor. Nécessite <code>@Transactional</code> pour le LAZY loading.
<code>addCourseToInstructor(instructorId, course)</code>	8j	Ajoute un nouveau Course à un Instructor existant.

8. Contrôleur REST — `TestSchoolController`

Le contrôleur TestSchoolController, annoté @RestController avec le préfixe /test/school, expose les méthodes du SchoolService sous forme d'API REST testables via Postman. Chaque méthode du service correspond à un endpoint HTTP.

HTTP	Endpoint	Annotation params	Méthode service
POST	/test/school/full	@RequestBody School	createSchoolWithDetails()
GET	/test/school/{id}	@PathVariable Long id	getSchoolById()
POST	/test/school/student?schoolId=X	@RequestBody Student, @RequestParam Long schoolId	createStudentWithAddressAndSchool()
GET	/test/school/students	(aucun)	listAllStudents()
POST	/test/school/instructor	@RequestBody Instructor	createInstructorWithCourses()
GET	/test/school/instructors/search?name=X	@RequestParam String name	listInstructorsByName()
GET	/test/school/instructor/{id}	@PathVariable Long id	getInstructorById()
GET	/test/school/course/{id}	@PathVariable Long id	getCourseById()
GET	/test/school/instructor/{id}/courses	@PathVariable Long id	getCoursesByInstructorId()
POST	/test/school/instructor/{id}/addCourse	@PathVariable Long id, @RequestBody Course	addCourseToInstructor()

Les annotations `@PathVariable` et `@RequestParam` remplissent deux rôles distincts. `@PathVariable` récupère une valeur directement intégrée dans le chemin de l'URL (ex: `/instructor/5` → `id=5`). `@RequestParam` récupère un paramètre transmis après le `?` dans l'URL (ex: `/search?name=Ali` → `name="Ali"`).

L'annotation `@RequestBody` permet à Spring de désérialiser automatiquement le corps JSON de la requête en objet Java. `@ResponseBody` permet de contrôler précisément le code HTTP retourné (200 OK, 404 Not Found, etc.).

9. Tests avec Postman

L'ensemble des endpoints REST a été testé à l'aide de Postman. Les corps JSON utilisés pour les requêtes POST sont présentés ci-dessous.

9.1 Créer une School complète — POST `/test/school/full`

```
{

  "name": "ISG Bizerte",

  "phone": "72123456",

  "departments": [

    { "name": "Informatique" },

    { "name": "Finance" }

  ],

  "instructors": [

    { "name": "Dr. Ben Ali" }

  ],

  "students": [

    {

      "firstName": "Ahmed", "lastName": "Trabelsi",

      "address": {

        "street": "Rue de la Paix", "city": "Bizerte", "zipCode": "7000"

      }

    }

  ]

}
```

Ce test valide la méthode `createSchoolWithDetails()` : une seule requête POST crée simultanément l'école, ses départements, ses instructeurs et ses étudiants avec leur adresse, grâce à la propagation en cascade.

9.2 Créer un Instructor avec Courses — POST `/test/school/instructor`

```
{

  "name": "Dr. Ben Ali",

  "courses": [

    { "name": "Java Avancée" },

    { "name": "Spring Boot" }

  ]

}
```

9.3 Créer un Student avec Address — POST /test/school/student?schoolId=1

```
{

  "firstName": "Sarrah",

  "lastName": "Mansour",

  "address": {

    "street": "Avenue Bourguiba", "city": "Tunis", "zipCode": "1000"

  }

}
```

9.4 Ajouter un Course à un Instructor existant — POST /test/school/instructor/1/addCourse

```
{ "name": "Architecture Logicielle" }
```

10. Technologies et frameworks utilisés

Technologie	Rôle dans le projet
Spring Boot	Framework principal. Fournit le serveur Tomcat embarqué, l'auto-configuration et le point d'entrée de l'application via <code>@SpringBootApplication</code> .
Spring Web (Spring MVC)	Gestion des requêtes HTTP. Fournit <code>@RestController</code> , <code>@GetMapping</code> , <code>@PostMapping</code> , <code>@PathVariable</code> , <code>@RequestParam</code> , <code>@RequestBody</code> .
Spring Data JPA	Couche d'abstraction sur JPA. Fournit <code>JpaRepository</code> avec CRUD automatique et génération de requêtes par nom de méthode.
JPA / Hibernate	Spécification et implémentation ORM. Gère le mapping objet-relationnel, génère le SQL, crée et met à jour les tables automatiquement.
Lombok	Génère le code répétitif (getters, setters, constructeurs) à la compilation via <code>@Data</code> , <code>@NoArgsConstructor</code> , <code>@AllArgsConstructor</code> .
Jackson	Sérialisation/désérialisation JSON automatique. Fournit <code>@JsonIgnore</code> pour éviter les boucles infinies de sérialisation.
PostgreSQL	Système de gestion de base de données relationnelle. Stocke les données persistantes dans la base <code>schooldb</code> .
Maven	Outil de build. Gère les dépendances via <code>pom.xml</code> , compile et package l'application.
Apache Tomcat	Serveur web embarqué. Reçoit les requêtes HTTP sur le port 8080, intégré automatiquement par Spring Boot.

11. Conclusion

Ce mini-projet nous a permis de mettre en pratique les concepts fondamentaux du développement Java d'entreprise, en développant un microservice complet de gestion scolaire selon les bonnes pratiques des architectures logicielles modernes.

L'analyse des différents cas de mapping JPA a mis en évidence l'importance du choix de la navigabilité : pour les relations OneToMany, le mapping bidirectionnel avec @JoinColumn côté Many est systématiquement préférable car il évite les tables de jointure superflues et offre une navigation dans les deux sens. Pour la relation OneToOne Student → Adress, le mapping unidirectionnel est le plus approprié car il respecte la logique métier (une adresse n'a pas besoin de connaître son étudiant) et la contrainte imposée par le sujet.

L'utilisation de Spring Data JPA simplifie considérablement l'accès aux données : les opérations CRUD sont disponibles sans écrire de SQL, et les requêtes dérivées comme findByNameContaining() sont générées automatiquement à partir du nom de la méthode. L'architecture en couches (Controller – Service – Repository) garantit une séparation claire des responsabilités et facilite la maintenance et l'évolution du projet.